

HipKittens × IRIS — Meeting 3

progress update: profiling the two collective ideas, and building QuantTile

Subha

8×MI350X (gfx950) · DeepSeek-R1 · HipKittens + IRIS

July 7, 2026

Since last time

- ▶ Last meeting was the fused MoE region: *1.56×* on prefill (1247 vs 1941 μ s, correctness-gated). Decode barely moved, and I said that was because decode is weight-bound, not boundary-bound.
- ▶ This week I stopped arguing from priors and measured. I got time on the dedicated 8×MI350X node and profiled the two comm-abstraction directions we keep circling back to.
- ▶ I also built *QuantTile* and validated it on that same node.
- ▶ Where it landed: the decode lever is *fewer weight bytes* (that's QuantTile, working today). The prefill lever is *fusing the all-reduce* (real ceiling, but the substrate to hit it isn't there yet).

What I profiled, and what came out

TP4, bf16, MI350X node. Unfused reference = a full-256-CU torch GEMM per rank's K-slice, then a separate RCCL all-reduce (what production does). Median-of-30, MAX over the 4 ranks (the slowest rank gates the collective).

GEMM (M, N, K) — R1 role	GEMM 256CU (ms)	RCCL AR (ms)	serial GEMM+AR (ms)	AR share	overlap ceiling (serial/AR)
8192,7168,18432 — down-proj	0.565	1.148	1.641	70%	1.43×
8192,7168,7168 — attn out-proj	0.252	1.147	1.357	85%	1.18×
8192,4608,36864 — example	0.606	0.750	1.269	59%	1.69×

The all-reduce is the expensive half

On prefill the AR is **59–85%** of the GEMM+AR it sits after, so comm isn't a rounding error next to the matmul. Hide the GEMM under it and the ceiling is **1.4–1.7×**.

Decode is different: DP-attention means **no TP all-reduce**, so nothing to fuse — it just streams all 32 experts' weights per step. Decode and prefill need different fixes.

Where HipKittens is today on quantized loads

HK hard-codes a *separate GEMM per quant format* — `kernels/gemm/fp8fp32` and `kernels/gemm/mx8fp8` are two different bodies. And fp8 can't load bytes straight to registers: the fast `buffer_load` path is a hard *`static_assert("Unsupported type for load")`*, so HK's fp8 kernels go the long way through LDS.

HK's fp8 decode path (staged through LDS)

global → shared (`buffer_load`)
→ register (`ds_read_b128`)
→ fp8 MFMA

I measured this LDS-staged path at *~2.6 TB/s* on the skinny BM=16 decode tile.

The LDS hop is the thing to cut

This is HK working as designed, not a bug. But that extra shared-memory hop is exactly what to skip when the tile is tiny and you're memory-bound. That's what sat skips (next slide).

What I built — the `sat` trick, and why it's faster

Keep the weights *compressed in HBM* (fp8 is half the bf16 bytes, fp4 a quarter), but load them through the *fast bf16 global→register path* instead of staging through LDS. Reinterpret the packed bytes as a half- or quarter-width bf16 tile, pre-swizzled offline into fragment order, unpack in-register, and hand a clean bf16 tile to a normal MFMA.

The sat path

```
global → register,  
bytes read as bf16 (buffer_load_b128)  
→ in-register unpack:  
fp8 — apply the per-block scale  
fp4 — hardware  
cvt_scalef32_pk_bf16_fp4  
→ bf16 MFMA
```

thor-4 (MI355X) anchor node	
weight path	throughput
HK fp8 (LDS-staged)	2.6 TB/s
<i>sat fp8</i> (bf16 reg path)	<i>3.97 TB/s</i>
<i>sat fp4</i>	<i>~1.6× over fp8</i>

This table is the thor-4 (MI355X) anchor — a faster box than the MI350X the rest of the deck runs on. I didn't re-verify it this week, but the ratios hold on both nodes. `sat fp8` also clears a dense bf16 load by *1.46×*; fp4's extra win comes from carrying *half the bytes of fp8*.

QuantTile — the primitive, and the validation

The two decode kernels I hand-wrote (fp8-sat, mxfp4-sat) are about *90% the same code*. They diverge in three spots: the packed-tile width, the scale layout, and the in-register unpack. That's a copy-paste fork waiting to rot.

- ▶ QuantTile makes the *format a compile-time property of the tile* — one descriptor carries width, scale layout, and unpack.
- ▶ So *one GEMM body* serves both fp8 and mxfp4, instead of the per-format fork HK ships today.

Validated on the MI350X node, identical inputs

- ▶ fp8: unified body within *0.07%* of the original — *2.874 TB/s* here, a wash. The abstraction is free.
- ▶ mxfp4: keeps *1.63×* over fp8.
- ▶ RMS matches the pre-unification kernels to *5 d.p.*

MXFP4: porting the fused region to 4-bit weights

amd/DeepSeek-R1-MXFP4 is OCP MXFP4 *W4A4* — fp4 weights *and* activations, group size 32, E8M0 scale per 32-element K-block. Route-1 keeps activations *bf16* (*weight-only fp4*): more accurate, and the right decode choice since decode is weight-bound.

fused decode region, 8×MI350, same node,

	weights	region (μ s)
TOTAL_M= 512	bf16	881.3
	fp8	578.9
	<i>MXFP4</i>	<i>520.6</i>

MXFP4: *1.11×* over fp8, *1.69×* over bf16.
Expert GEMM *alone* is 1.16× / 1.96×; the region dilutes it with the shared act-dequant and the gather/act/combine boundaries. FFN
PASS, RMS 0.018.

The GEMM already exists — the work is the MX wiring

Reuses HK's mxfp8 8-wave scaled-MFMA body (`mma_ABt_scaled` already reads the MX per-32-block E8M0 scale), wrapped with our expert-grouping and the IRIS gather/-combine.

Honest caveat: no fp4-vs-SOTA number yet — aiter's a4w4 EP path is untuned/buggy for our shape. The trusted baseline is the SGLang R1-MXFP4 e2e run (being set up); fp4's activation-quant win is really a *prefill* effect, weight-only fp4 is the decode variant.

The other lever — fusing the prefill all-reduce

The prefill idea is to fold the TP4 all-reduce *into* the GEMM: reduce output tiles as they're produced, so the short GEMM hides under the long all-reduce. Slide 3 says the ceiling is real, *1.4–1.7×*. This is the tiles-before-reduce idea — how many tiles you accumulate before you reduce — as one knob.

Honest status: the substrate isn't ready

The IRIS fused-collective examples I actually ran are *4.7×* to *280×* *slower* than plain unfused today. They're pedagogical — a slow Triton GEMM plus a naive in-kernel AR.

What it would take

A real HK producer/consumer GEMM body at torch rate whose epilogue emits tile transfers, plus an in-kernel reduce-scatter at RCCL bandwidth. Both are substantial, and it's *prefill-only*. Not done — this is the harder direction.

The communication abstraction: `tile_reduce_scatter`

The combine hand-writes a loop of `iris.store` calls: each destination token locally reduces its up-to-8 rows in *fp32* (weighted, no cross-rank atomics), writes one bf16 row home, over a hand-rolled order of tiles across the 8 links.

Declare the transfer, the library lowers it

Declares a `TileTransferSet` (destination map, a CSR grouping of each tile's source rows, the `src/weight/dst` payload, the IRIS handle) and calls `tile_reduce_scatter`. The library (`tilecomm_device.h`) does the `fp32` reduce, the `ctx.store` home (local-write short-circuit), and *owns the store order* — a bad hand-written schedule can't happen. No hand-written `iris.load/store`.

Why this shape

The *residency-axis* analog of `QuantTile` (the format axis): declare the intent, the library lowers it. Bulk-synchronous first — a standalone combine kernel, a low-risk drop-in; the *same* declaration later goes inside the GEMM tile loop for overlap. Store granularity is a knob (2 / 4 / 16 B).

Status: code written, combine refactored onto it (original body kept for a same-build A/B). *Not yet profiled* — GPU on the `QuantTile`-vs-SOTA run; the bar is *zero cost* (June-30 combine 386 μ s vs MORI's 398 μ s). *Gap*: fire-and-forget stores need a real completion primitive.

How the two fit together

Both are the same trick on the tile: QuantTile touches *format*, the collective touches *residency*. Two lowerings of the same tile.

the tile carries...	lowers to...	regime
<i>FORMAT</i> (QuantTile)	in-register unpack + bf16 MFMA	decode weight wall
<i>RESIDENCY</i> (collective)	in-kernel reduce-scatter over IRIS	prefill all-reduce

I'm not pretending these get tuned by one knob — format and residency stay two separate problems. They just share the tile descriptor.

Positioning: the residency side is a general tile-level RMA collective over IRIS — reusable for both the MoE gather/combine *and* the TP reduce-scatter/all-reduce — not a claim to be first at MoE communication (DeepEP and NCCL-EP already exist). The bet is the abstraction, not the transport.

Open questions for the experts

This room knows distributed algorithms better than I do. Seven things I'd like to argue about — questions I'm bringing, not answers I have:

1. *Completion.* What proves a device-initiated remote store has landed and is visible — kernel exit, a fence, an atomic release, a readback, an SHMEM-style *quiet*? Ours look fire-and-forget.
2. *Which algorithm.* For our TP4 message sizes on 8 fully-connected MI350X, what wins — direct all-push, ring, recursive-halving/doubling, or whatever RCCL picks?
3. *Relay routing.* Under expert skew, can 2-hop relay beat direct point-to-point once you pay injection, relay, and sync — or is the fabric already saturated?
4. *Deadlock.* What makes in-kernel arrival-counter waits provably deadlock-free when not all producer blocks are even resident yet?
5. *Scheduling model.* What's the right formal model for finite-wave tile scheduling with issue queues and uneven tile sizes?
6. *Batching.* Does “accumulate G tiles, then reduce” still hold for IRIS stores over XGMI, or does async completion change the latency term?
7. *Novelty.* Next to MSCCL++, SCCL, NCCL-EP, and DeepEP — which framing would you accept: the API, the schedule ownership, the memory-model safety, or the fused tile execution?